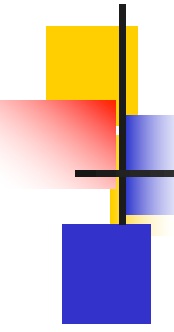


Support Vector Machines (SVM) and How to Do It in R



History of SVM

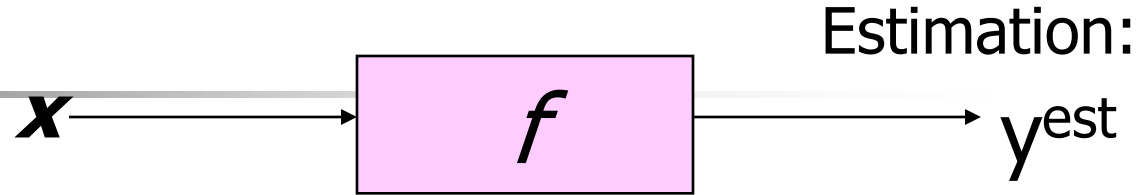
- SVM is related to the statistical learning theory [3]
- SVM was first introduced in 1992 [1]
- SVM became popular on the strength of its success in handwritten digit recognition
 - 1.1% test error rate for SVM. This is the same as the error rates of a carefully constructed neural network, LeNet 4.
 - See Section 5.11 in [2] or the discussion in [3] for details
- SVM is now regarded as an important example of “kernel methods”, one of the **key areas in machine learning**
 - Note: the meaning of “kernel” is different from the “kernel” function for Parzen windows

[1] B.E. Boser *et al.* A Training Algorithm for Optimal Margin Classifiers. Proceedings of the Fifth Annual Workshop on Computational Learning Theory 5 144-152, Pittsburgh, 1992.

[2] L. Bottou *et al.* Comparison of classifier methods: a case study in handwritten digit recognition. Proceedings of the 12th IAPR International Conference on Pattern Recognition, vol. 2, pp. 77-82.

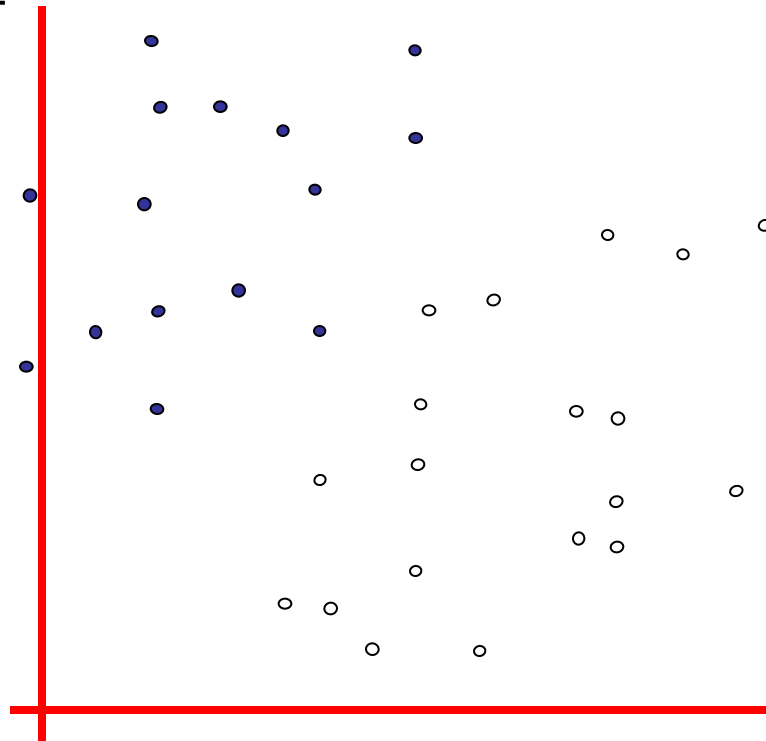
[3] V. Vapnik. The Nature of Statistical Learning Theory. 2nd edition, Springer, 1999.

Linear Classifiers



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

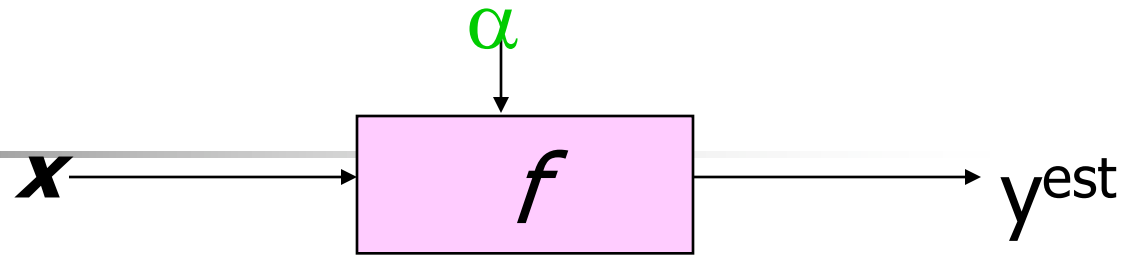
- denotes +1
- denotes -1



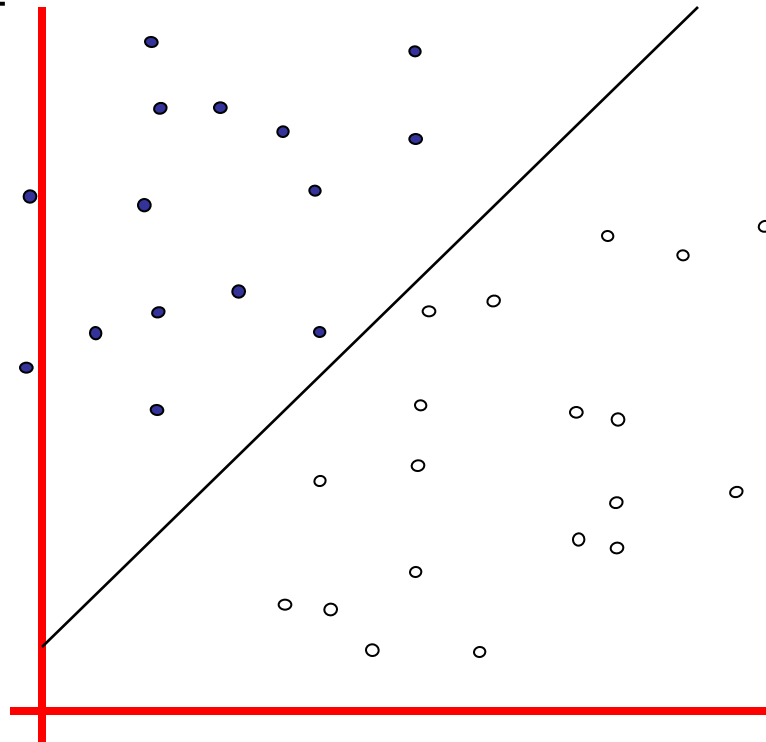
w: weight vector
x: data vector

How would you
classify this data?

Linear Classifiers



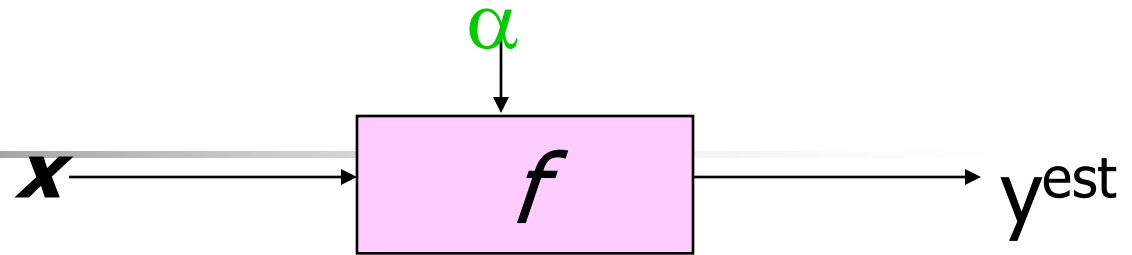
- denotes +1
- denotes -1



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

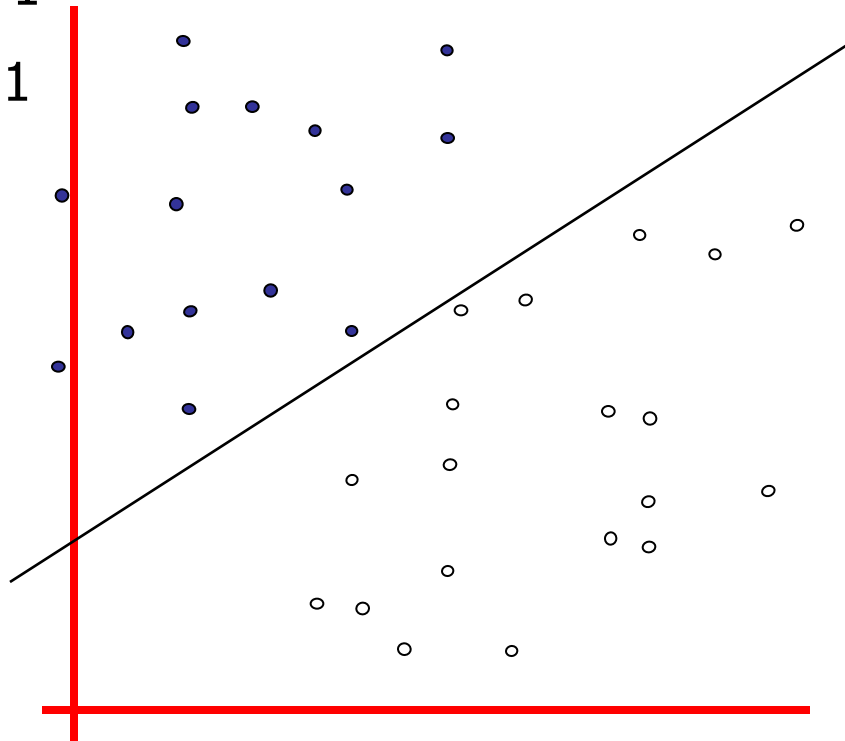
How would you classify this data?

Linear Classifiers



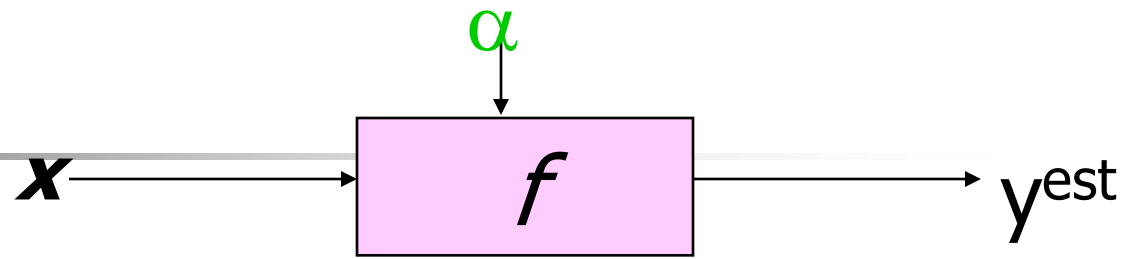
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

- denotes +1
- denotes -1

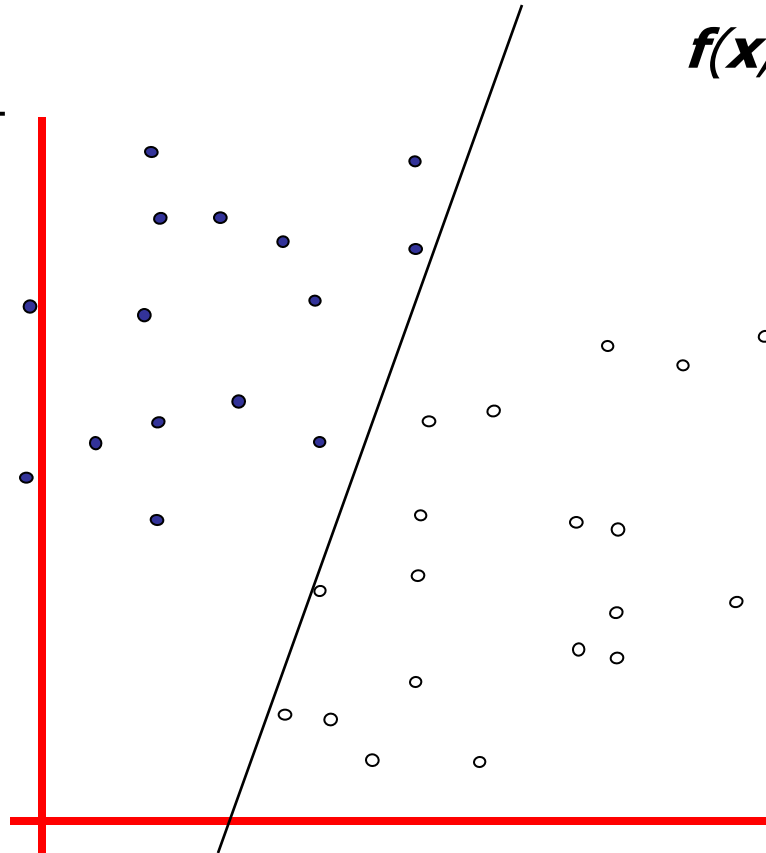


How would you classify this data?

Linear Classifiers



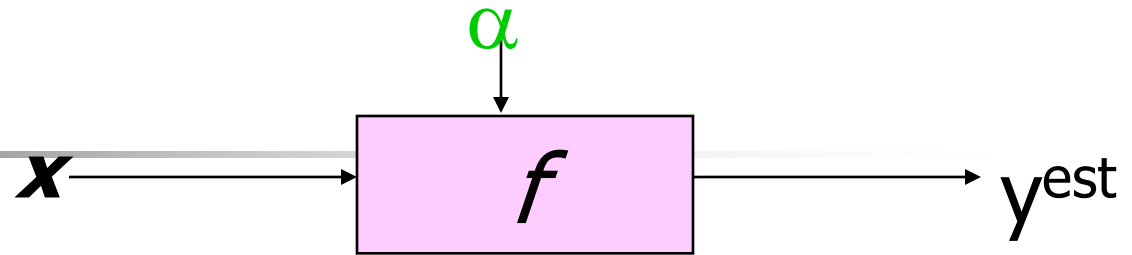
- denotes +1
- denotes -1



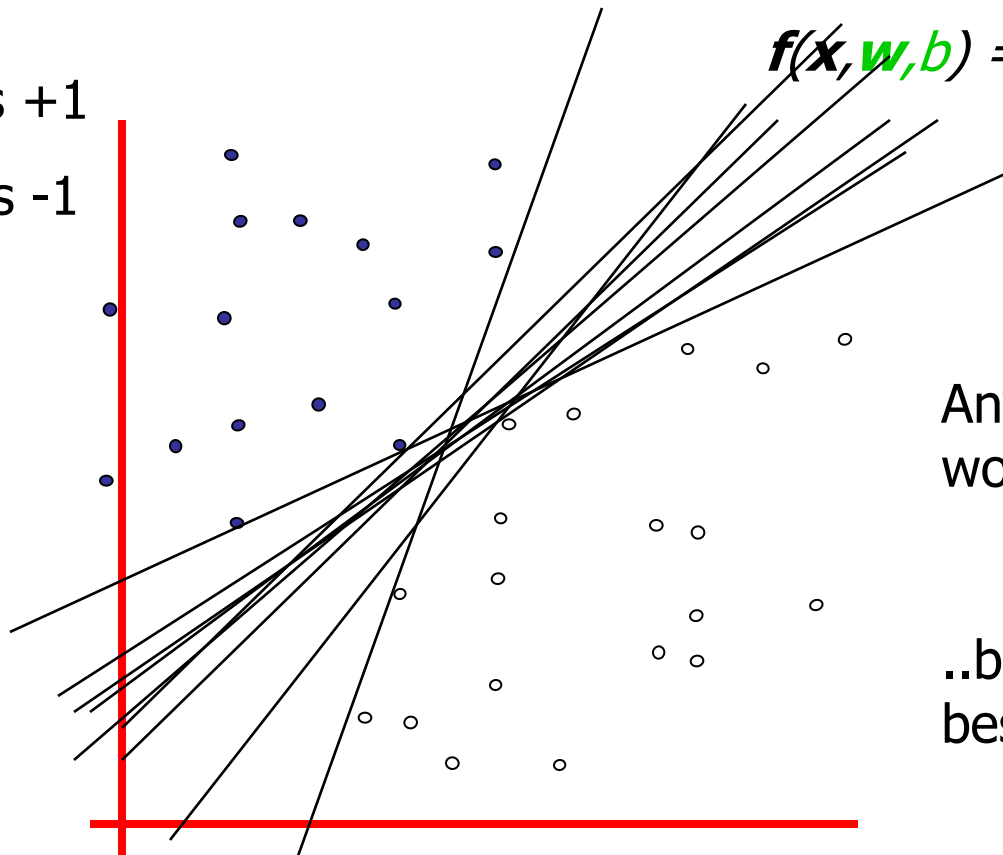
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

How would you classify this data?

Linear Classifiers



- denotes +1
- denotes -1

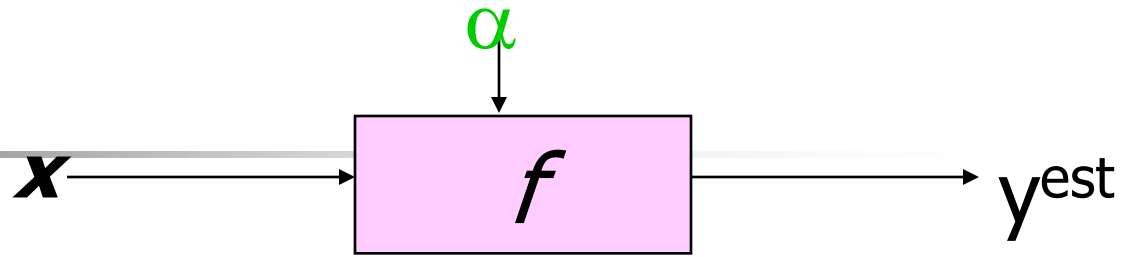


$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

Any of these
would be fine..

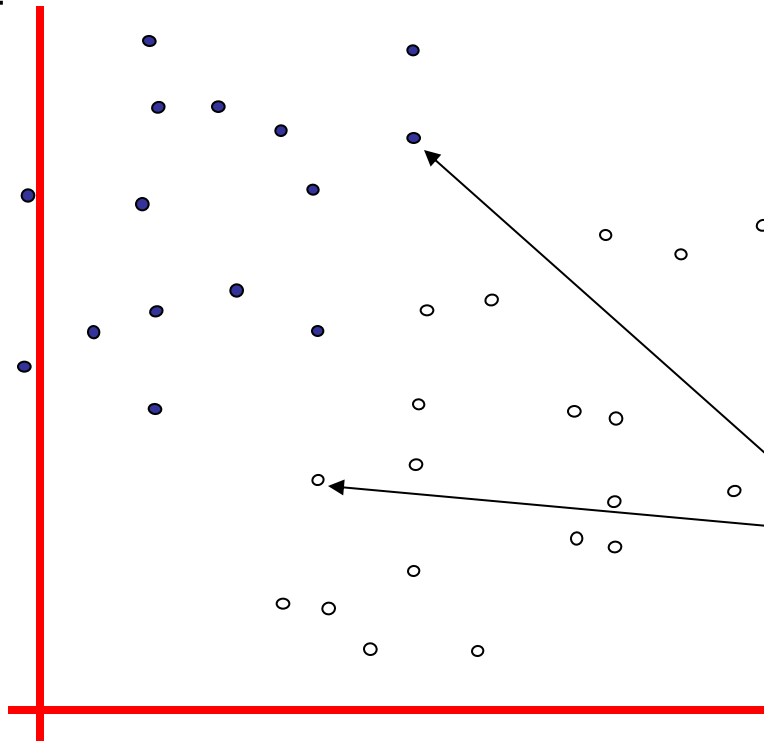
..but which is
best?

Classifier Margin



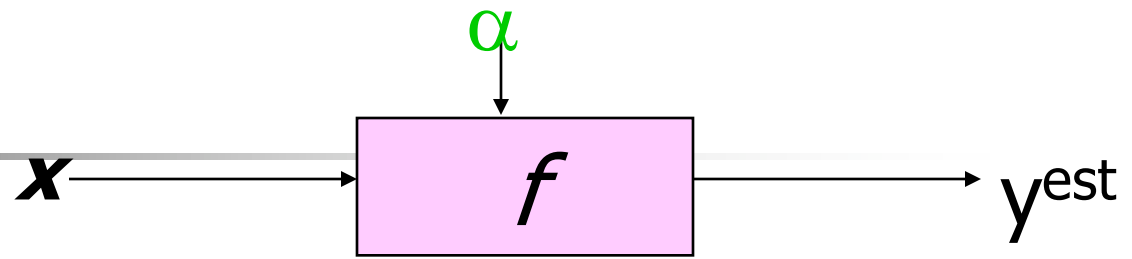
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

- denotes +1
- denotes -1

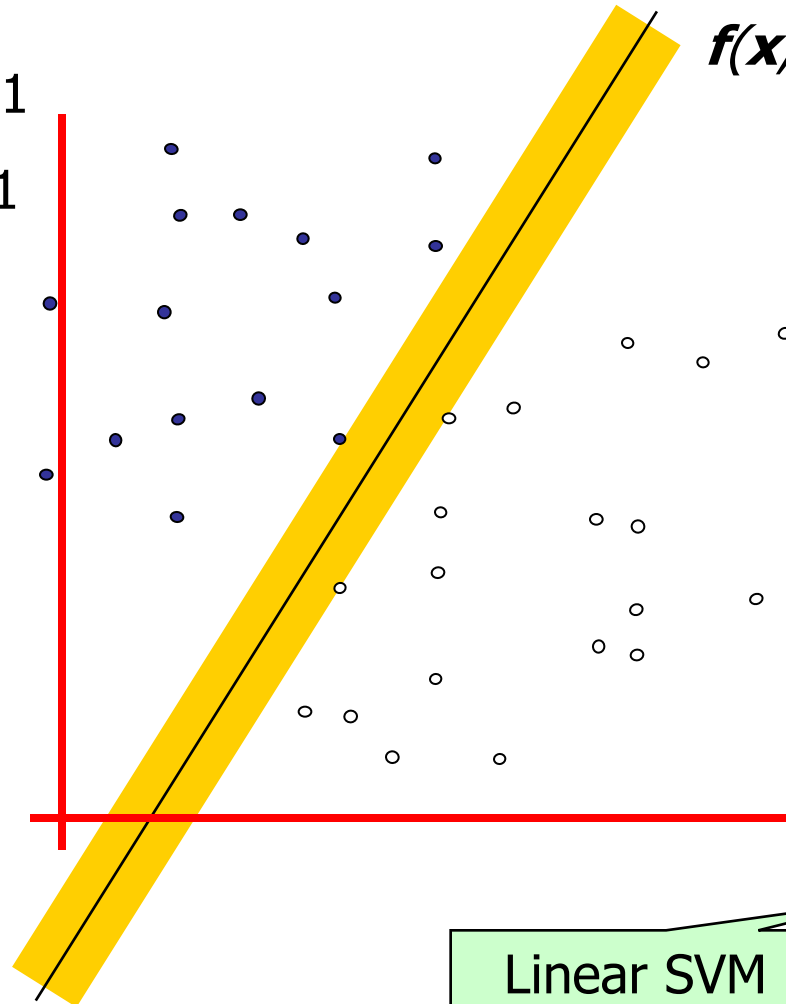


Define the **margin** of a linear classifier as the width that the boundary could be increased by **before hitting a datapoint.**

Maximum Margin



- denotes +1
- denotes -1



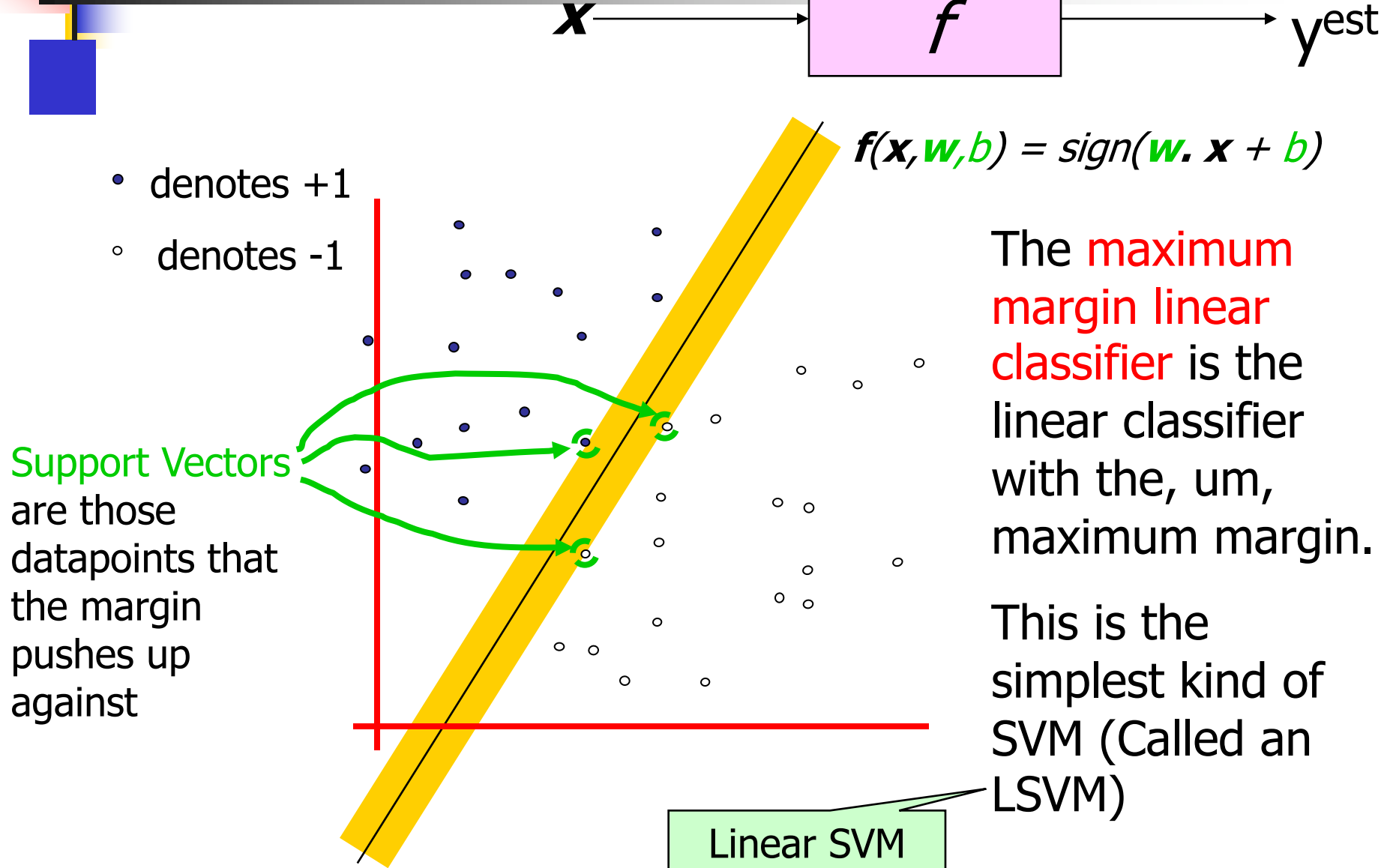
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

This is the simplest kind of SVM (Called an LSVM)

Linear SVM

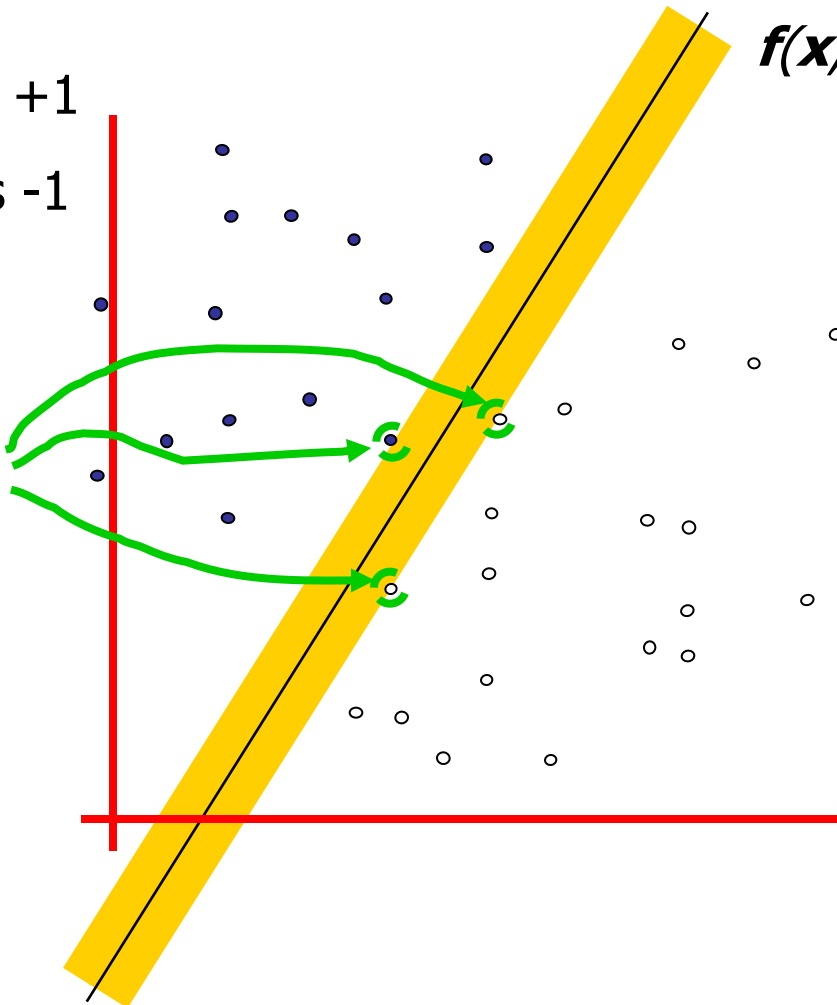
Maximum Margin



Why Maximum Margin?

- denotes +1
- denotes -1

Support Vectors
are those
datapoints that
the margin
pushes up
against

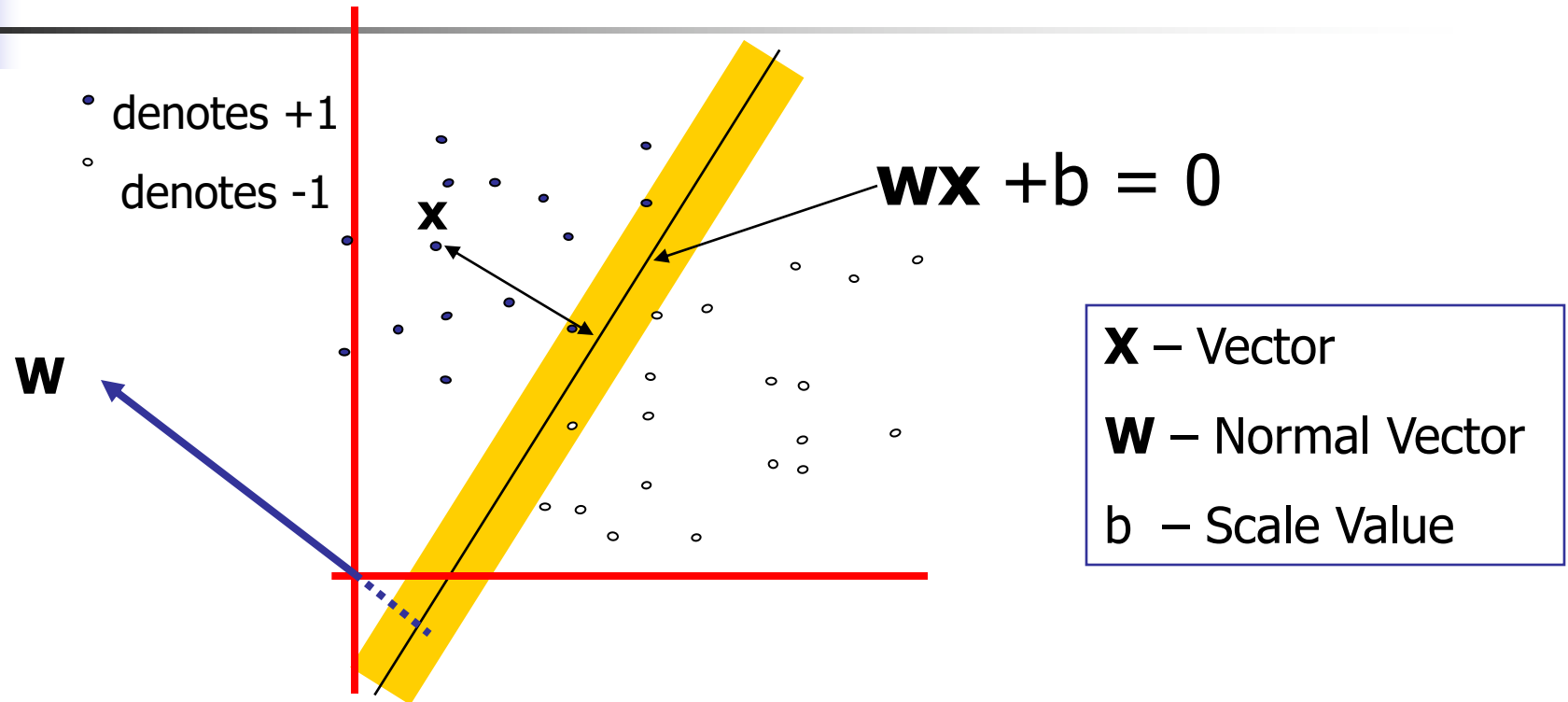


$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

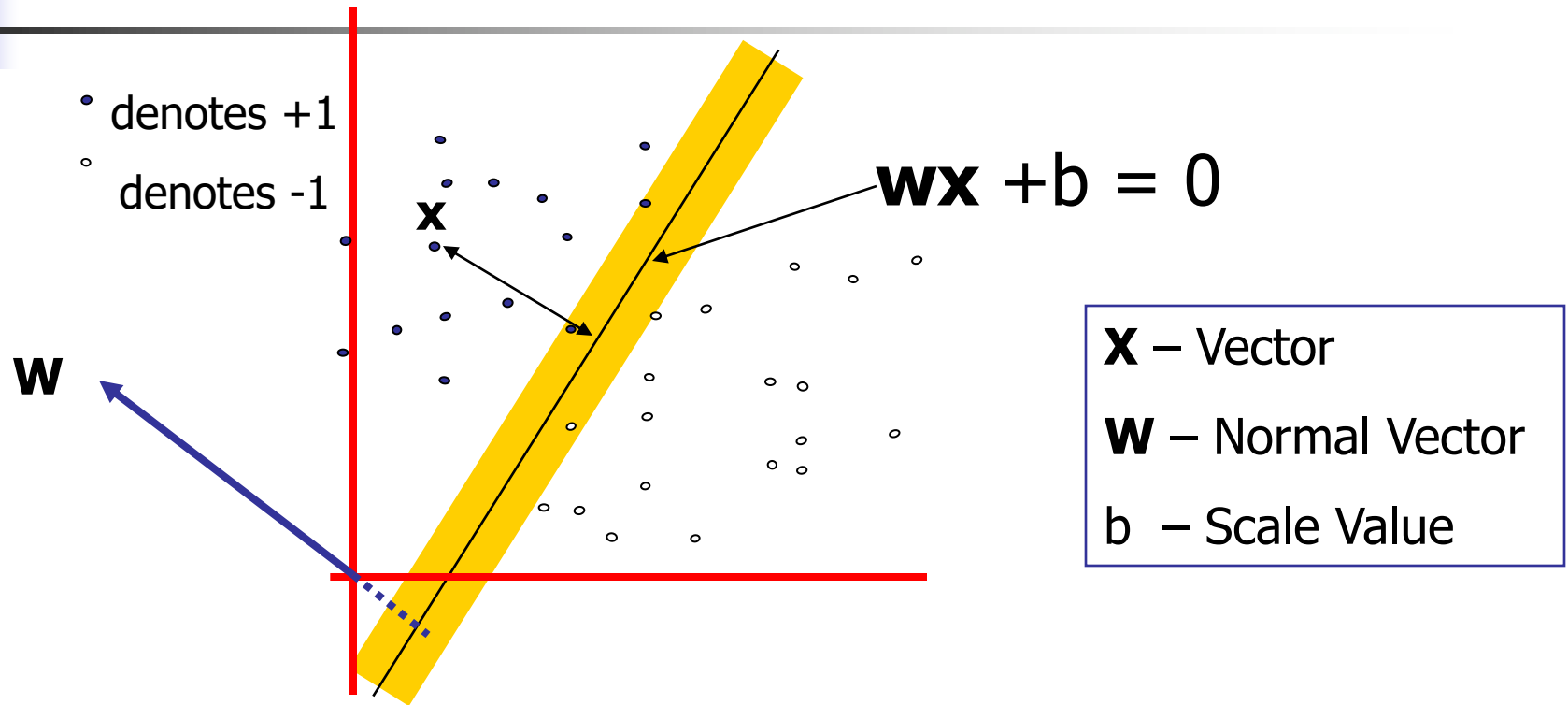
This is the simplest kind of SVM (Called an LSVM)

How to calculate the distance from a point to a line?



- <http://mathworld.wolfram.com/Point-LineDistance2-Dimensional.html>
- In our case, $w_1 * x_1 + w_2 * x_2 + b = 0$,
- thus, $\mathbf{w} = (w_1, w_2)$, $\mathbf{x} = (x_1, x_2)$

Estimate the Margin

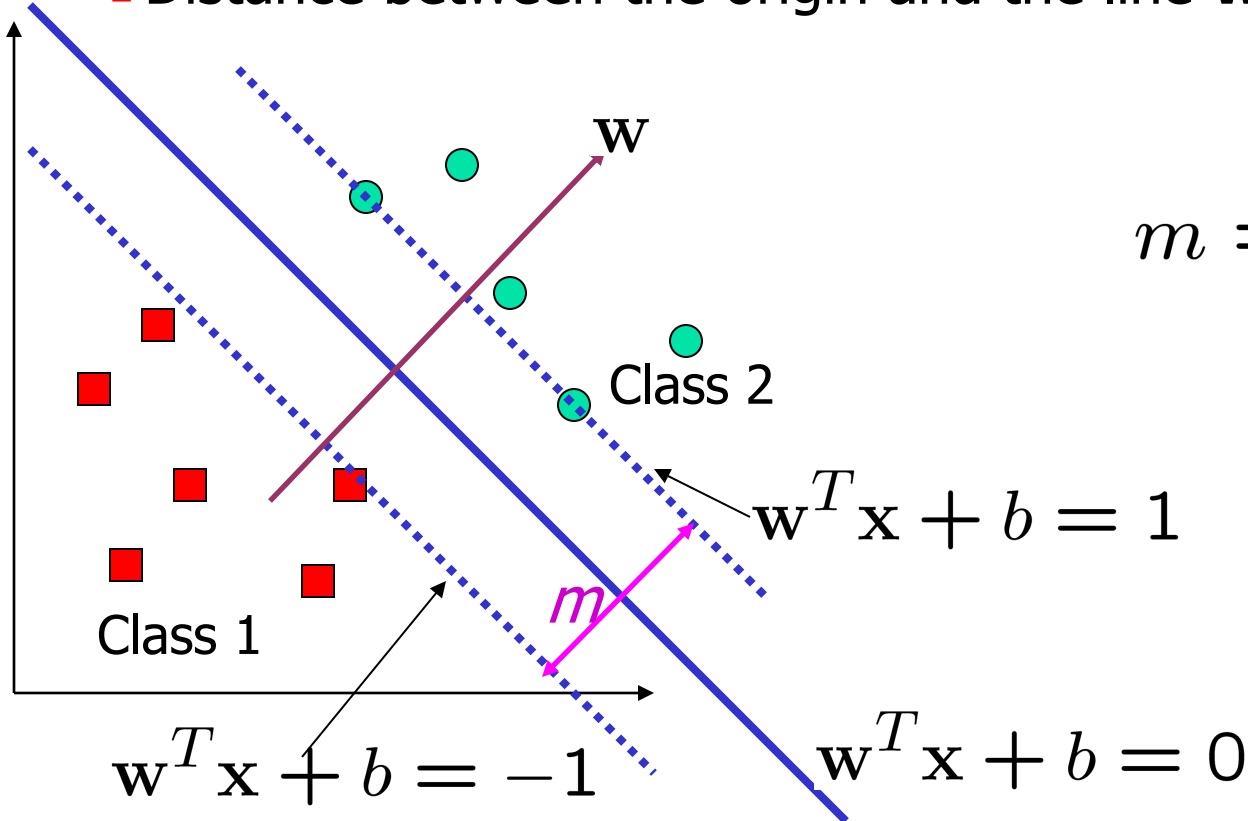


- What is the distance expression for a point $\mathbf{x}$ to a line $\mathbf{w}\mathbf{x} + b = 0$?

$$d(\mathbf{x}) = \frac{|\mathbf{x} \cdot \mathbf{w} + b|}{\sqrt{\|\mathbf{w}\|_2^2}} = \frac{|\mathbf{x} \cdot \mathbf{w} + b|}{\sqrt{\sum_{i=1}^d w_i^2}}$$

Large-margin Decision Boundary

- The decision boundary should be as far away from the data of both classes as possible
 - We should maximize the margin, m
 - Distance between the origin and the line $\mathbf{w}^T \mathbf{x} = -b$ is $b/||\mathbf{w}||$



$$m = \frac{2}{||\mathbf{w}||}$$

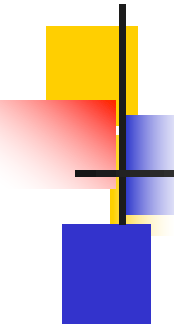
Finding the Decision Boundary

- Let $\{x_1, \dots, x_n\}$ be our data set and let $y_i \in \{1, -1\}$ be the class label of x_i
- The decision boundary should classify all points correctly
 $\Rightarrow y_i(w^T x_i + b) \geq 1, \quad \forall i$
- To see this: when $y=-1$, we wish $(wx+b)<-1$;
when $y=1$, we wish $(wx+b)>1$.

For support vectors, we wish $y(wx+b)=1$.

- The decision boundary can be found by solving the following constrained optimization problem

$$\begin{aligned} & \text{Minimize } \frac{1}{2} \|w\|^2 \\ & \text{subject to } y_i(w^T x_i + b) \geq 1 \quad \forall i \end{aligned}$$



Next step... Optional

- Converting SVM to a form we can solve
 - Dual form
- Allowing a few errors
 - Soft margin
- Allowing nonlinear boundary
 - Kernel functions

The Dual Problem (we ignore the derivation)

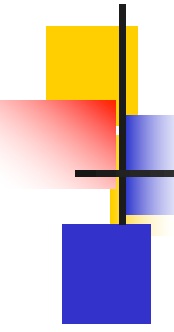
- The new objective function is in terms of α_i only
- It is known as the dual problem: if we know $\mathbf{w}$, we know all α_i ; if we know all α_i , we know $\mathbf{w}$
- The original problem is known as the primal problem
- The objective function of the dual problem needs to be maximized!
- The dual problem is therefore:

$$\max. \quad W(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{subject to } \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0$$

Properties of α_i when we introduce the Lagrange multipliers

The result when we differentiate the original Lagrangian w.r.t. $\mathbf{b}$



The Dual Problem

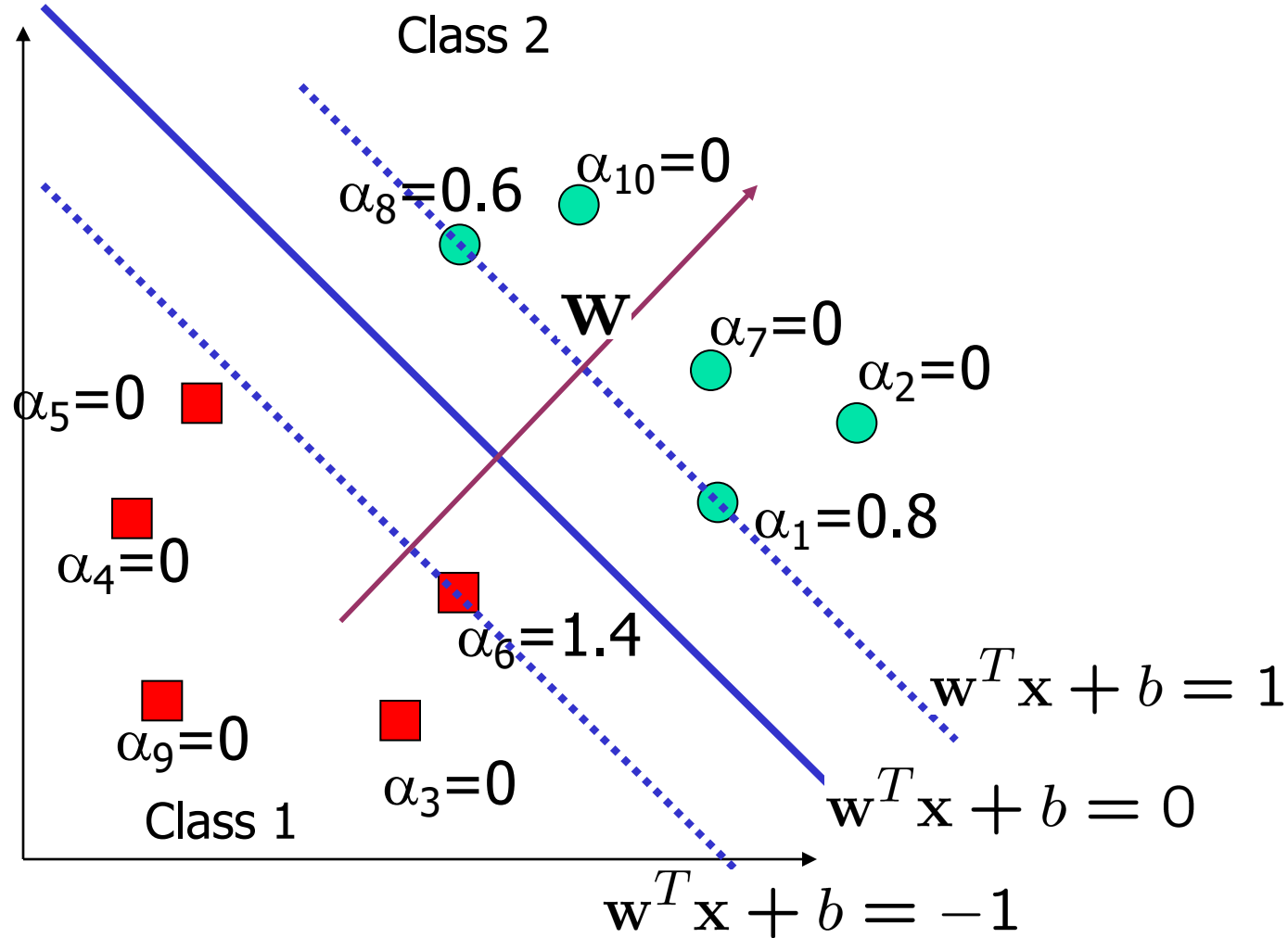
$$\begin{aligned} \max. \quad W(\boldsymbol{\alpha}) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to } \alpha_i &\geq 0, \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

- This is a quadratic programming (QP) problem
 - A global maximum of α_i can always be found
- $\mathbf{w}$ can be recovered by $\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$

Characteristics of the Solution

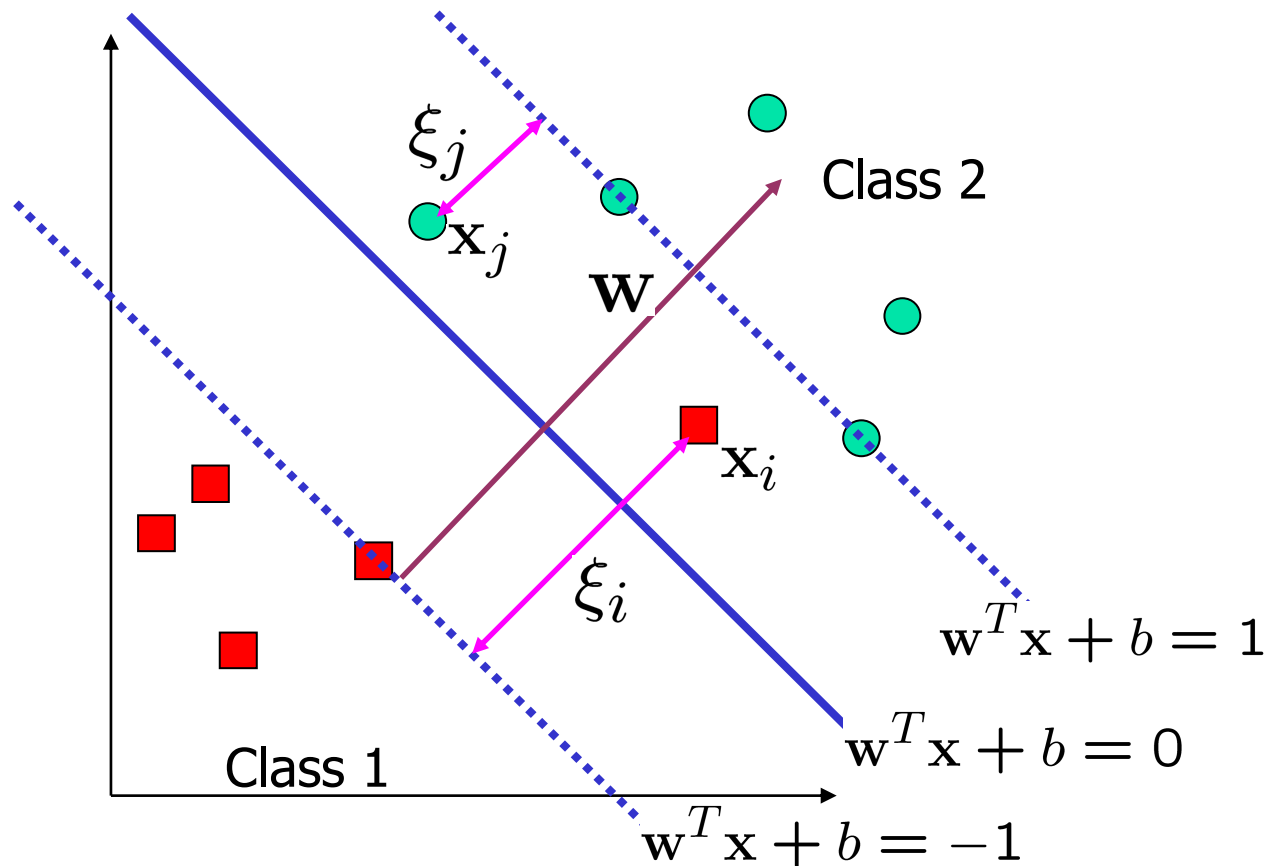
- Many of the α_i are zero (see next page for example)
 - $\mathbf{w}$ is a linear combination of a small number of data points
 - This “sparse” representation can be viewed as data compression as in the construction of knn classifier
- $\mathbf{x}_i$ with non-zero α_i are called support vectors (SV)
 - The decision boundary is determined only by the SV
 - Let t_j ($j=1, \dots, s$) be the indices of the s support vectors.
We can write
$$\mathbf{w} = \sum_{j=1}^s \alpha_{t_j} y_{t_j} \mathbf{x}_{t_j}$$
- For testing with a new data $\mathbf{z}$
 - Compute $\mathbf{w}^T \mathbf{z} + b = \sum_{j=1}^s \alpha_{t_j} y_{t_j} (\mathbf{x}_{t_j}^T \mathbf{z}) + b$ and classify $\mathbf{z}$ as class 1 if the sum is positive, and class 2 otherwise
 - Note: $\mathbf{w}$ need not be formed explicitly

A Geometrical Interpretation



Allowing errors in our solutions

- We allow “error” ξ_i in classification; it is based on the output of the discriminant function $\mathbf{w}^T \mathbf{x} + b$
- ξ_i approximates the number of misclassified samples

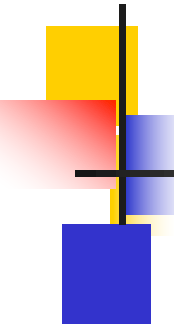


Soft Margin Hyperplane

- If we minimize $\sum_i \xi_i$, ξ_i can be computed by

$$\begin{cases} \mathbf{w}^T \mathbf{x}_i + b \geq 1 - \xi_i & y_i = 1 \\ \mathbf{w}^T \mathbf{x}_i + b \leq -1 + \xi_i & y_i = -1 \\ \xi_i \geq 0 & \forall i \end{cases}$$

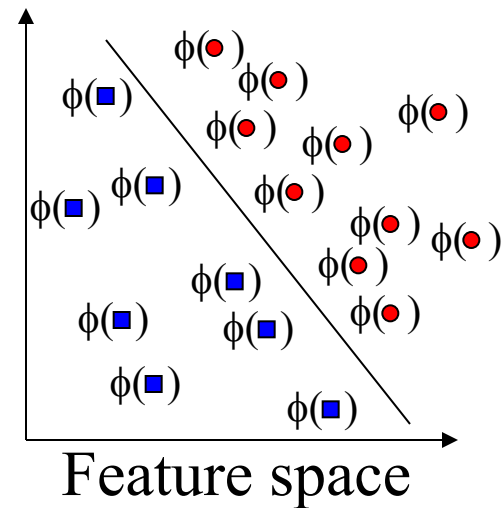
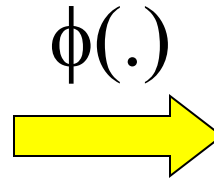
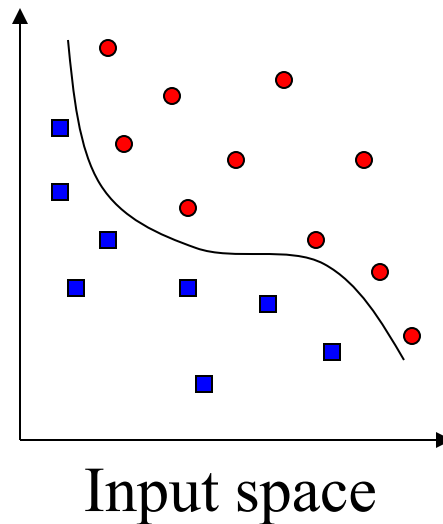
- ξ_i are “slack variables” in optimization
- Note that $\xi_i=0$ if there is no error for $\mathbf{x}_i$
- ξ_i is an upper bound of the number of errors
- **We want to minimize** $\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$
 - **C (Cost)**: tradeoff parameter between error and margin
- The optimization problem becomes
 - Minimize $\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$
 - subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$



Extension to Non-linear Decision Boundary

- So far, we have only considered large-margin classifier with a linear decision boundary
- How to generalize it to become nonlinear?
- Key idea: transform $\mathbf{x}_i$ to a higher dimensional space to “make life easier”
 - Input space: the space the point $\mathbf{x}_i$ are located
 - Feature space: the space of $\phi(\mathbf{x}_i)$ after transformation

Transforming the Data (c.f. DHS Ch. 5)



Note: feature space is of higher dimension than the input space in practice

- Computation in the feature space can be costly because it is high dimensional
 - The feature space is typically infinite-dimensional!
- The kernel trick comes to rescue

The Kernel Trick

- Recall the SVM optimization problem

$$\begin{aligned} \max. \quad W(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to } C &\geq \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

- The data points only appear as **inner product**
- As long as we can calculate the inner product in the feature space, we do not need the mapping explicitly
- Many common geometric operations (angles, distances) can be expressed by inner products
- Define the kernel function K by

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

An Example for $\phi(\cdot)$ and $K(\cdot, \cdot)$

- Suppose $\phi(\cdot)$ is given as follows

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

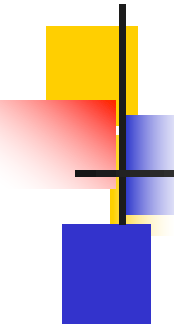
- An inner product in the feature space is

$$\langle \phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right), \phi\left(\begin{bmatrix} y_1 \\ y_2 \end{bmatrix}\right) \rangle = (1 + x_1y_1 + x_2y_2)^2$$

- So, if we define the kernel function as follows, there is no need to carry out $\phi(\cdot)$ explicitly

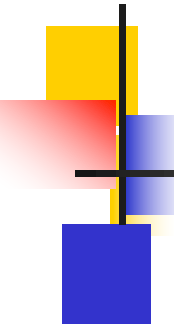
$$K(\mathbf{x}, \mathbf{y}) = (1 + x_1y_1 + x_2y_2)^2$$

- This use of kernel function to avoid carrying out $\phi(\cdot)$ explicitly is known as the **kernel trick**



More on Kernel Functions

- Not all similarity measures can be used as kernel function, however
 - The kernel function needs to satisfy the **Mercer function**, i.e., the function is “positive-definite”
https://en.wikipedia.org/wiki/Definite_matrix
- This implies that
 - the n by n kernel matrix,
 - in which the (i,j) -th entry is the $K(\mathbf{x}_i, \mathbf{x}_j)$, is always positive definite
- This also means that optimization problem can be solved in polynomial time!



Examples of Kernel Functions

- Polynomial kernel with degree d

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^d$$

- Radial basis function kernel with width σ

$$K(\mathbf{x}, \mathbf{y}) = \exp(-\|\mathbf{x} - \mathbf{y}\|^2 / (2\sigma^2))$$

- Closely related to radial basis function neural networks
- The feature space is infinite-dimensional

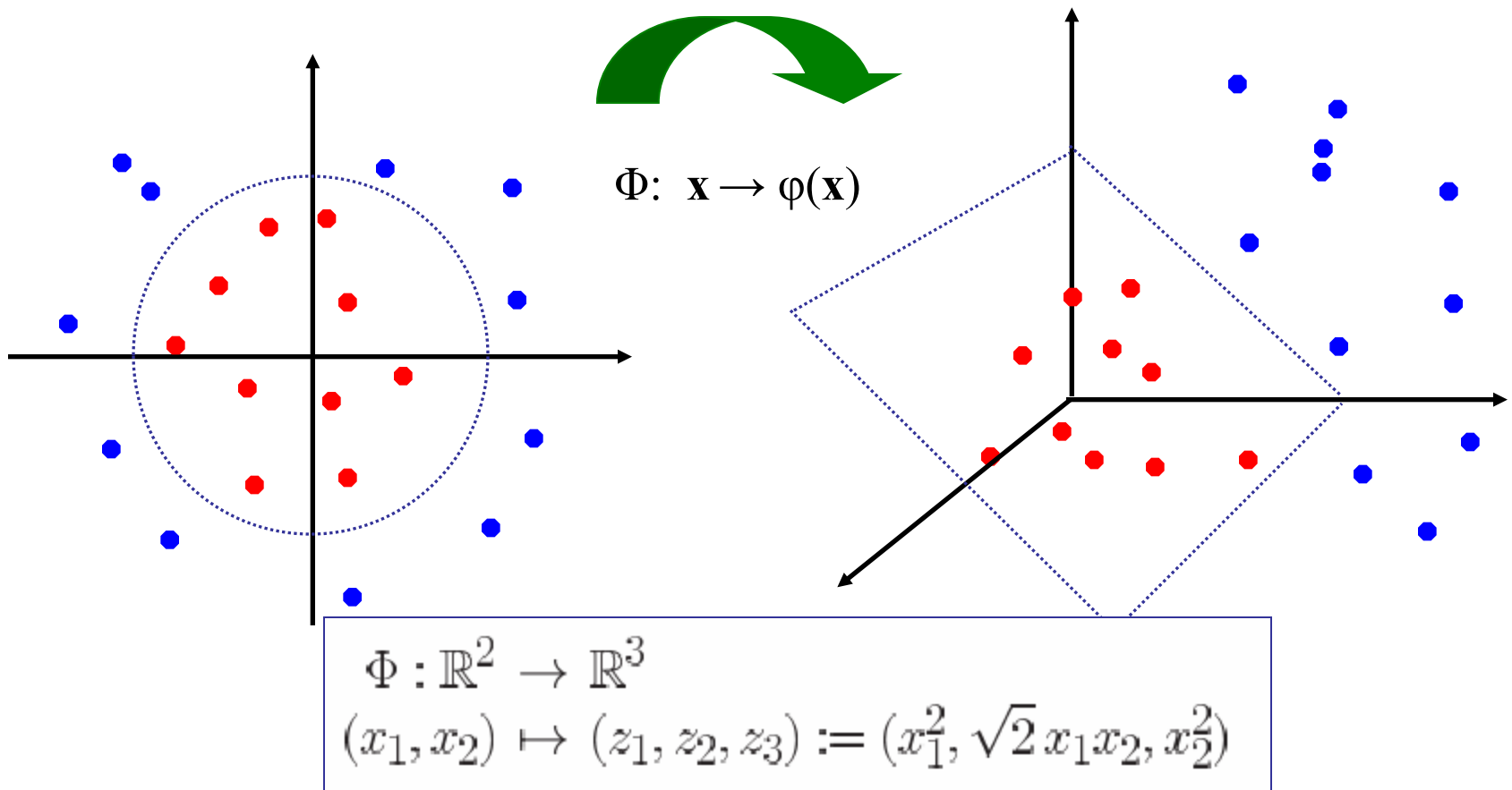
- Sigmoid with parameter κ and θ

$$K(\mathbf{x}, \mathbf{y}) = \tanh(\kappa \mathbf{x}^T \mathbf{y} + \theta)$$

- It does not satisfy the Mercer condition on all κ and θ

Non-linear SVMs: Feature spaces

General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



Example

- Suppose we have 5 one-dimensional data points
 - $x_1=1, x_2=2, x_3=4, x_4=5, x_5=6$, with 1, 2, 6 as class 1 and 4, 5 as class 2 $\Rightarrow y_1=1, y_2=1, y_3=-1, y_4=-1, y_5=1$
- We use the polynomial kernel of degree 2
 - $K(x,y) = (xy+1)^2$
 - C is set to 100
- We first find α_i ($i=1, \dots, 5$) by

$$\begin{aligned} \max. \quad & \sum_{i=1}^5 \alpha_i - \frac{1}{2} \sum_{i=1}^5 \sum_{j=1}^5 \alpha_i \alpha_j y_i y_j (x_i x_j + 1)^2 \\ \text{subject to} \quad & 100 \geq \alpha_i \geq 0, \sum_{i=1}^5 \alpha_i y_i = 0 \end{aligned}$$

Example

- By using a QP solver, we get
 - $\alpha_1=0, \alpha_2=2.5, \alpha_3=0, \alpha_4=7.333, \alpha_5=4.833$
 - Note that the constraints are indeed satisfied
 - The support vectors are $\{x_2=2, x_4=5, x_5=6\}$

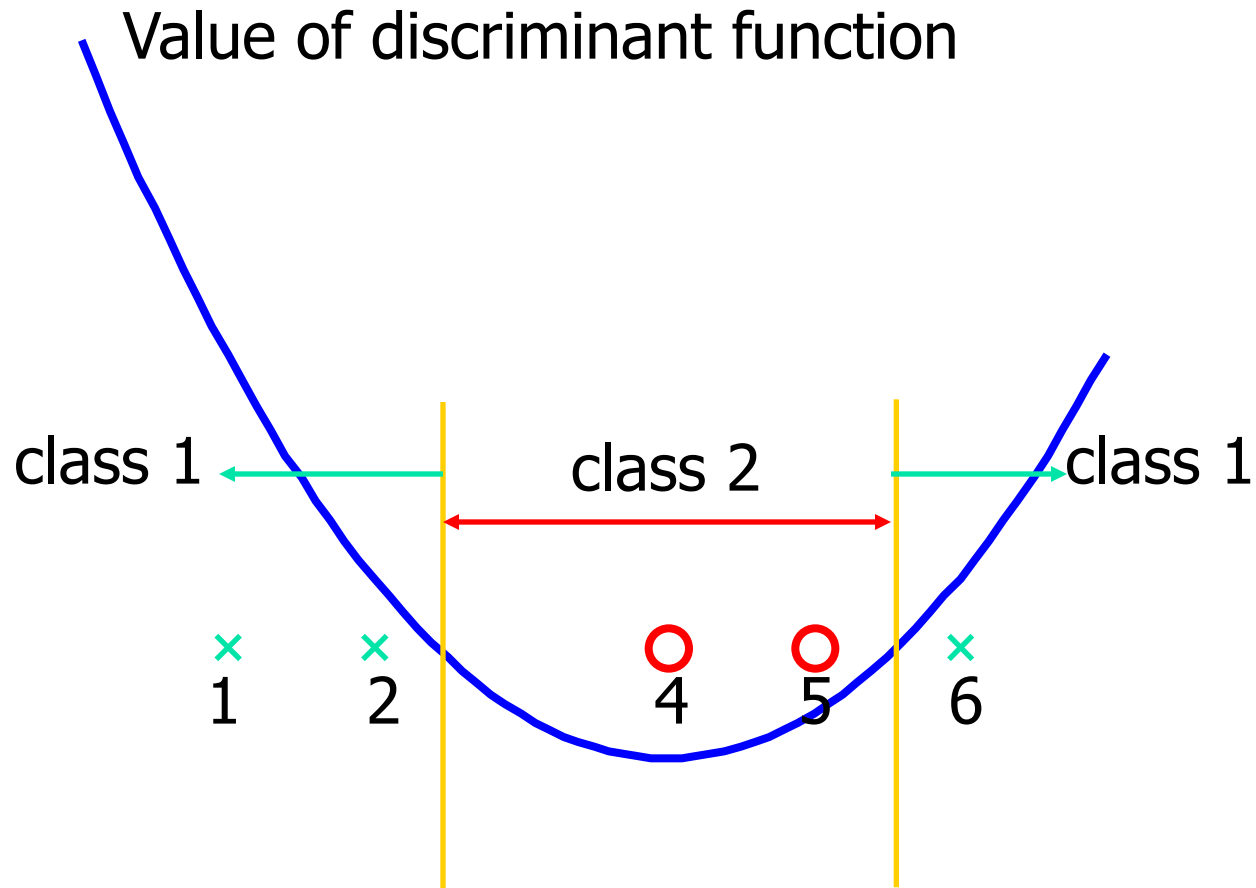
- The discriminant function is

$$\begin{aligned} f(z) &= 2.5(1)(2z + 1)^2 + 7.333(-1)(5z + 1)^2 + 4.833(1)(6z + 1)^2 + b \\ &= 0.6667z^2 - 5.333z + b \end{aligned}$$

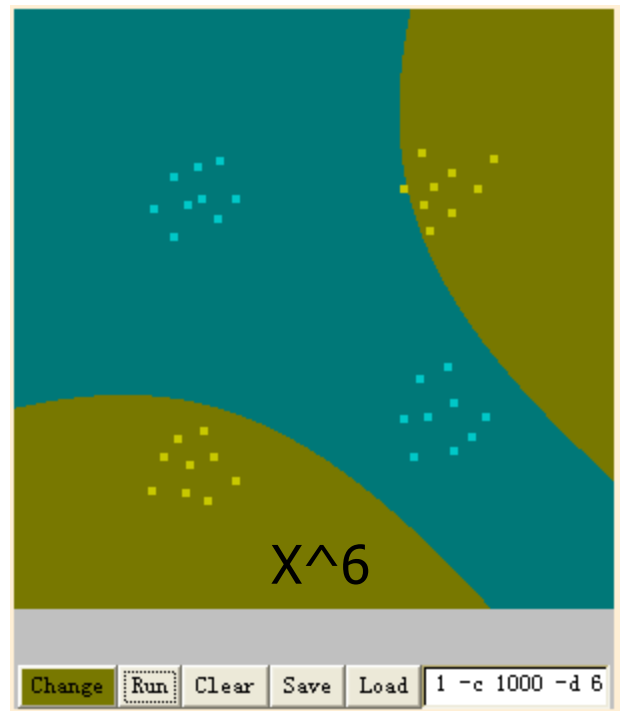
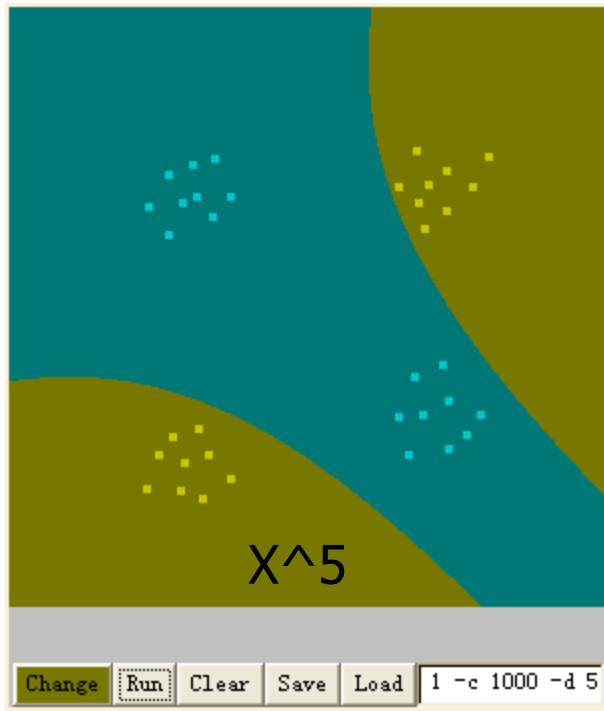
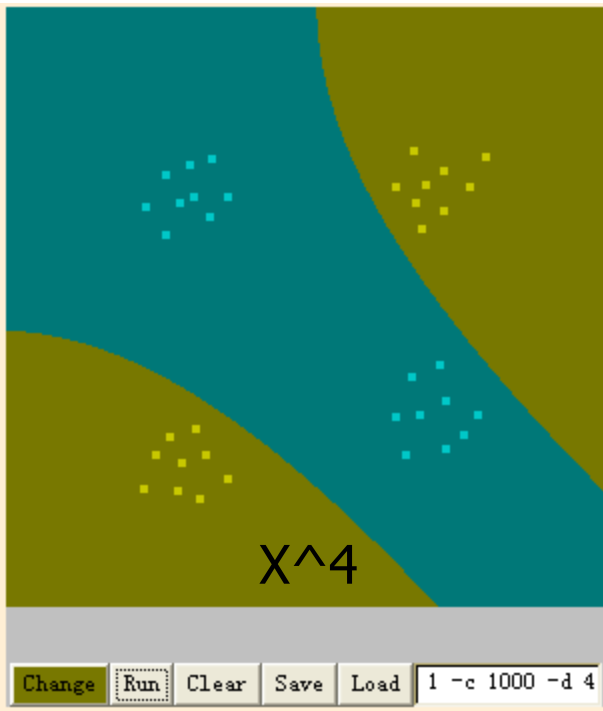
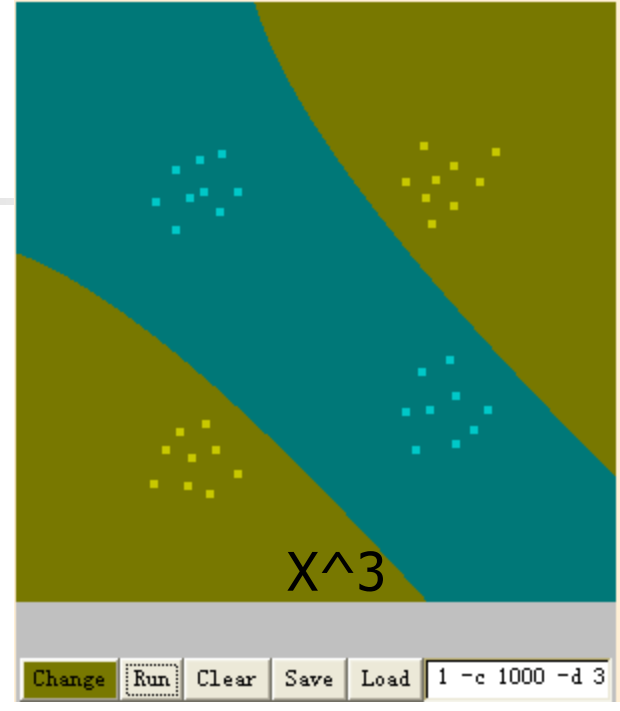
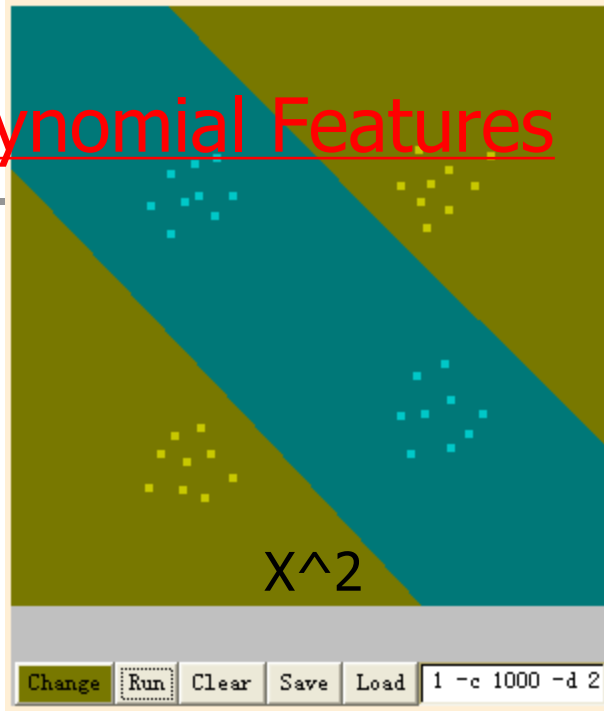
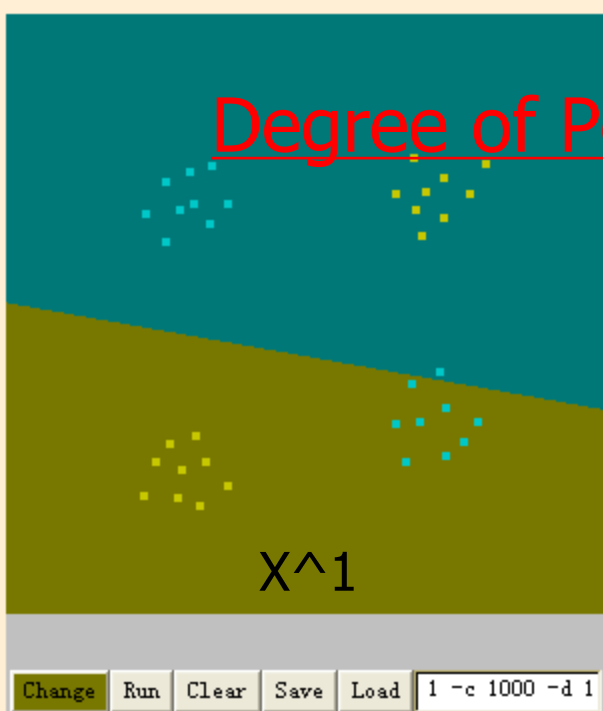
α_5 y_5 $K(z, x_5)$

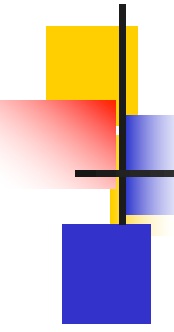
- b is recovered by solving $f(2)=1$ or by $f(5)=-1$ or by $f(6)=1$, as x_2 and x_5 lie on the line $\phi(w)^T \phi(x) + b = 1$ and x_4 lies on the line $\phi(w)^T \phi(x) + b = -1$
- All three give $b=9 \rightarrow f(z) = 0.6667z^2 - 5.333z + 9$

Example



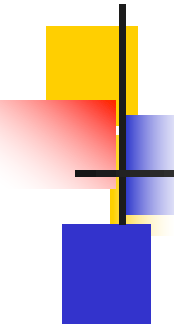
Degree of Polynomial Features





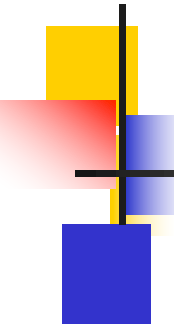
Choosing the Kernel Function

- Probably the most tricky part of using SVM.



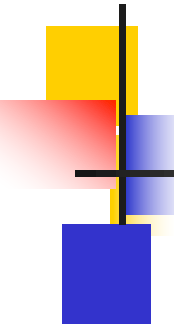
Software

- A list of SVM implementation can be found at <http://www.kernel-machines.org/software.html>
- Some implementation (such as LIBSVM) can handle multi-class classification
- SVMLight is among one of the earliest implementation of SVM
- Several Matlab toolboxes for SVM are also available



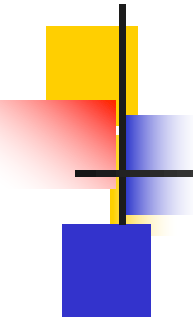
Summary: Steps for Classification

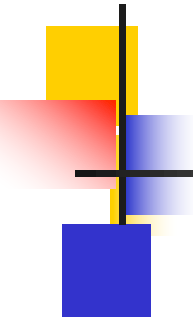
- Prepare the pattern matrix
- Select the kernel function to use
- Select the parameter of the kernel function and the value of C
 - You can use the values suggested by the SVM software, or you can set apart a validation set to determine the values of the parameter
- Execute the training algorithm and obtain the α_i
- Unseen data can be classified using the α_i and the support vectors



Conclusion

- SVM is a useful alternative to neural networks
- Two key concepts of SVM: maximize the margin and the kernel trick
- Many SVM implementations are available on the web for you to try on your data set!

- 
- 
-
- Now we shall learn how to perform SVM in R



SVM in R

- caret:

<http://www.sthda.com/english/articles/36-classification-methods-essentials/144-svm-model-support-vector-machine-essentials/>

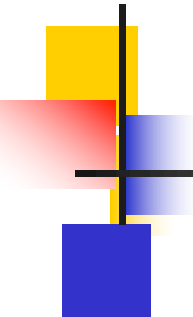
<https://www.youtube.com/watch?v=RKZoJVMr6CU>

- e1071:

http://rstudio-pubs-static.s3.amazonaws.com/497469_f0ec684e3cd34eb28b1e8a4989a00c22.html

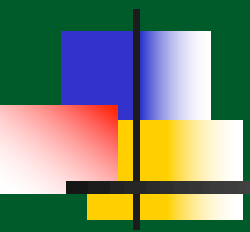
<https://www.datacamp.com/community/tutorials/support-vector-machines-r>

<https://www.youtube.com/watch?v=NOXCy4Aj4CE>



Resources

- <http://www.kernel-machines.org/>
- <http://www.support-vector.net/>
- <http://www.support-vector.net/icml-tutorial.pdf>
- <http://www.kernel-machines.org/papers/tutorial-nips.ps.gz>
- <http://www.clopinet.com/isabelle/Projects/SVM/applist.html>



*Thanks:
Prof. Andrew Moore
CMU*

<http://www.cs.cmu.edu/~awm/tutorials>

&

*Prof. Martin Law
Michigan State University*